

Padrões de Projeto (Facade)

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Facade
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação



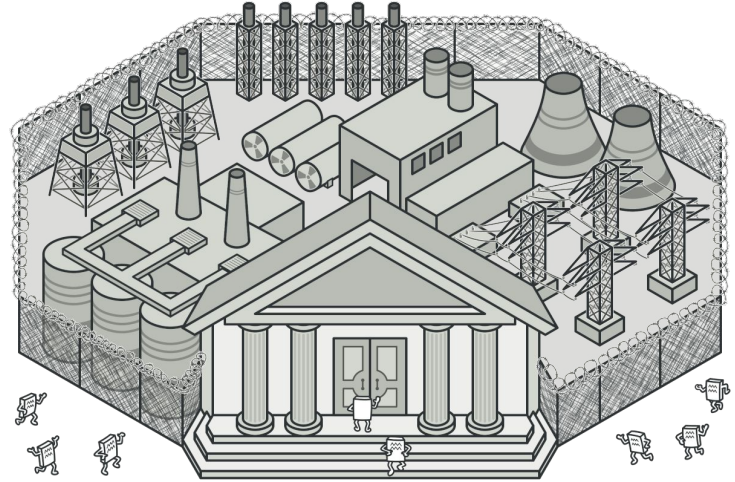
Facade

Introdução

- O Facade (ou Fachada) é um padrão de projeto estrutural que fornece uma interface simplificada para acessar sistemas, bibliotecas ou frameworks complexos.
- Ele atua como um intermediário que reduz o acoplamento entre o cliente e os detalhes internos de um subsistema.

Padrão Facade

- O padrão oculta a complexidade de um conjunto de classes e fornece um ponto único de acesso.
- O cliente interage apenas com a fachada, sem precisar conhecer a estrutura interna ou as dependências entre os componentes do sistema.



Problema

- Imagine que você precisa fazer seu código funcionar com um amplo conjunto de objetos que pertencem a uma sofisticada biblioteca ou framework. Normalmente, você precisaria inicializar todos aqueles objetos, rastrear as dependências, executar métodos na ordem correta, e assim por diante.
- Como resultado, a lógica de negócio de suas classes vai ficar firmemente acoplada aos detalhes de implementação das classes de terceiros, tornando difícil compreendê-lo e mantê-lo.

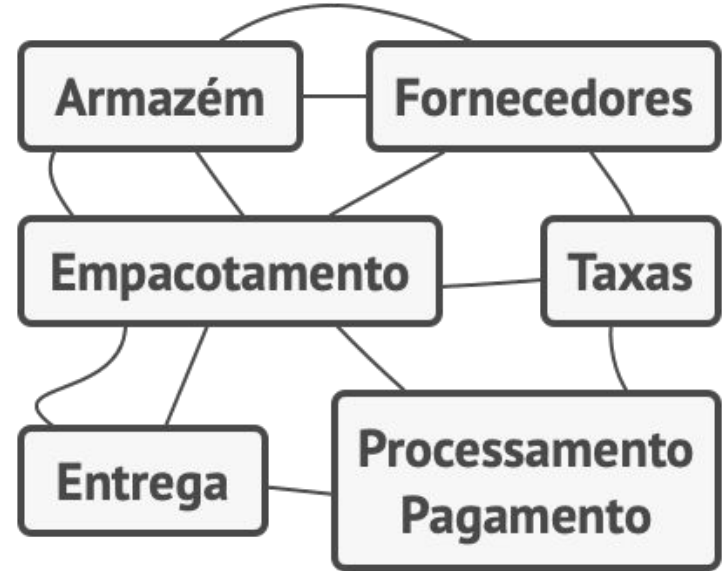
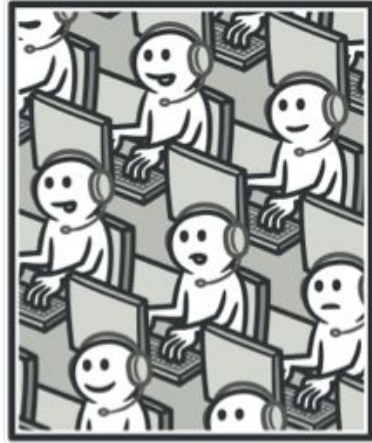
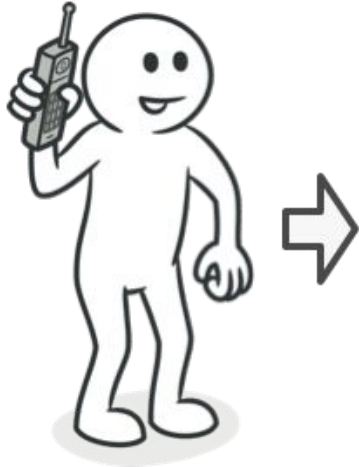
Solução

- Criar uma classe fachada que encapsula e organiza as interações com o subsistema.
- Ela oferece apenas as funcionalidades essenciais de forma simplificada, enquanto a lógica complexa permanece escondida.
- Assim, o cliente trabalha com poucos métodos de alto nível e não precisa lidar com detalhes internos.

Analogia do Mundo Real

- Ao ligar para uma loja para fazer um pedido, você fala com o operador, que representa a fachada.
- Ele coordena pedidos, pagamentos e entregas — você não precisa contatar cada departamento diretamente.
- A fachada faz o mesmo com sistemas de software complexos.

Analogia do Mundo Real

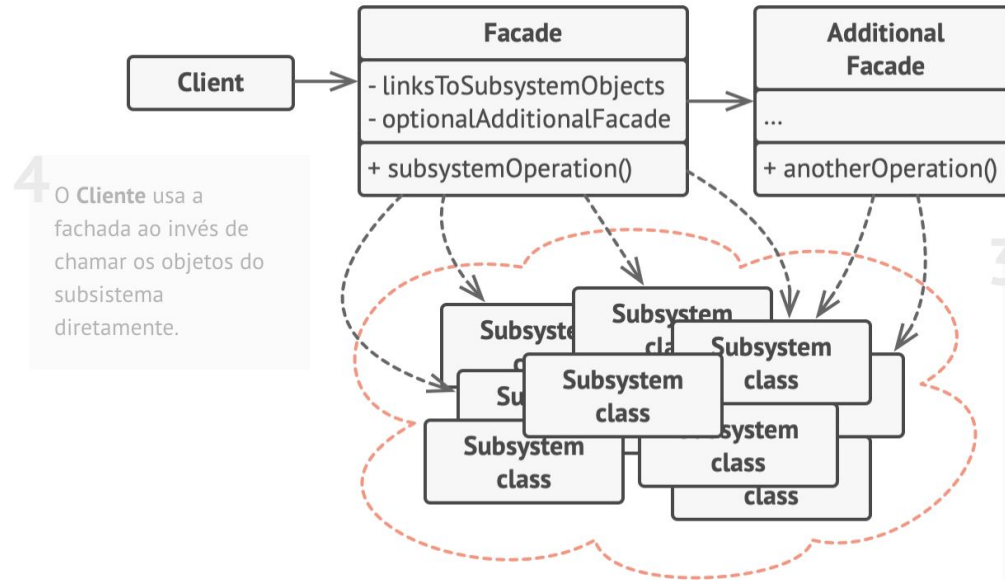


Estrutura do Padrão

- **Facade (Fachada):** fornece a interface simples e coordena chamadas para o subsistema.
- **Subsystem (Subsistema):** contém a lógica complexa e múltiplas classes que executam o trabalho real.
- **Cliente:** utiliza a fachada, sem precisar conhecer ou interagir diretamente com o subsistema.
- **Fachadas adicionais:** podem ser criadas para organizar subsistemas por área funcional (ex.: áudio, vídeo, rede).

1 A **Fachada** fornece um acesso conveniente para uma parte particular da funcionalidade do subsistema. Ela sabe onde direcionar o pedido do cliente e como operar todas as partes móveis.

2 Uma classe **Fachada Adicional** pode ser criada para prevenir a poluição de uma única fachada com funcionalidades não relevantes que podem torná-lo mais uma estrutura complexa. Fachadas adicionais podem ser usadas tanto por clientes como por outras fachadas.



4 O **Cliente** usa a fachada ao invés de chamar os objetos do subsistema diretamente.

3 O **Subsistema Complexo** consiste em dúzias de objetos variados. Para tornar todos eles em algo que signifique alguma coisa, você tem que mergulhar fundo nos detalhes de implementação do subsistema, tais como objetos de inicialização na ordem correta e supri-los com dados no formato correto.

As classes do subsistema não estão cientes da existência da fachada. Elas operam dentro do sistema e trabalham entre si diretamente.

```
// Subsistema complexo
class CPU {
    fun freeze() = println("CPU: congelando processador...")
    fun jump(position: String) = println("CPU: saltando para a posição $position...")
    fun execute() = println("CPU: executando instruções...")
}
```

```
class Memory {
    fun load(position: String, data: String) {
        println("Memória: carregando dados '$data' na posição $position...")
    }
}
```

```
class HardDrive {
    fun read(sector: String): String {
        println("Disco Rígido: lendo setor $sector...")
        return "dados do setor $sector"
    }
}
```

```
// Fachada
class ComputerFacade(
    private val cpu: CPU = CPU(),
    private val memory: Memory = Memory(),
    private val hardDrive: HardDrive = HardDrive()
) {
    fun startComputer() {
        println("Iniciando o computador...")
        cpu.freeze()
        val data = hardDrive.read("A1")
        memory.load("0x00", data)
        cpu.jump("0x00")
        cpu.execute()
        println("Computador iniciado com sucesso!")
    }
}
```

```
// Cliente
fun main() {
    val computer = ComputerFacade()
    computer.startComputer()
}
```

Exemplo em Kotlin

```
# Subsistema complexo
class CPU:
    def freeze(self):
        print("CPU: congelando processador...")

    def jump(self, position):
        print(f"CPU: saltando para a posição {position}...")

    def execute(self):
        print("CPU: executando instruções...")

class Memory:
    def load(self, position, data):
        print(f"Memória: carregando dados '{data}' na posição {position}...")

class HardDrive:
    def read(self, sector):
        print(f"Disco Rígido: lendo setor {sector}...")
        return f"dados do setor {sector}"

# Fachada que simplifica o uso do subsistema
class ComputerFacade:
    def __init__(self):
        self.cpu = CPU()
        self.memory = Memory()
        self.hard_drive = HardDrive()

    def start_computer(self):
        print("Iniciando o computador...")
        self.cpu.freeze()
        data = self.hard_drive.read("A1")
        self.memory.load("0x00", data)
        self.cpu.jump("0x00")
        self.cpu.execute()
        print("Computador iniciado com sucesso!")

# Cliente usa apenas a fachada
if __name__ == "__main__":
    computer = ComputerFacade()
    computer.start_computer()
```

Exemplo em Python

Aplicabilidade

- **Use o Facade quando:**
 - Você deseja simplificar o acesso a um sistema ou biblioteca complexa.
 - Precisa desacoplar o código cliente de dependências de terceiros.
 - Deseja organizar sistemas em camadas, criando um ponto de entrada para cada módulo (ex.: rede, persistência, interface).

Prós e Contras

- **Vantagens**

- Simplifica o uso de sistemas complexos.
- Reduz o acoplamento entre cliente e subsistema.
- Facilita substituições de frameworks (alterando apenas a fachada).

- **Desvantagens**

- Pode se tornar um “objeto deus” se concentrar muita lógica.
- Ocultar demais pode limitar o acesso a funcionalidades específicas do subsistema.



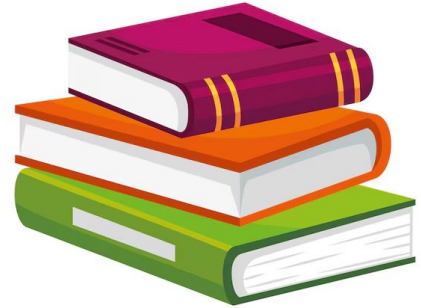
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

